UMassAmherst | Manning College of Information & Computer Sciences

Programming Methodology

Lab 6: Property-Based Testing and Observer Pattern

Wednesday March 11, 2026

Weekly Lab Agenda

- Go over reminders/goals
- Review past material
- Work in groups of 2-3 to solve a few exercises
    - Please sit with your group from last week.
- Discussion leaders will walk around and answer questions
- Solutions to exercises will be reviewed as a class
- Attendance taken at the end

Reminders

- Homework 5 is due Wednesday (3/11) at 11:59pm
    - Come to office hours for help!
- Midterm 1 is being graded. Aiming to finish grading by Friday.
    - Exam solutions will also be posted later this week.
- Submit what you have at the end of lab to Gradescope. If you miss many submissions to Gradescope, we may penalize your lab grade.

Today's Goals

- Property-Based Testing
- Observer Pattern

Property-Based Testing

Used when a problem has **more than one** right answer.

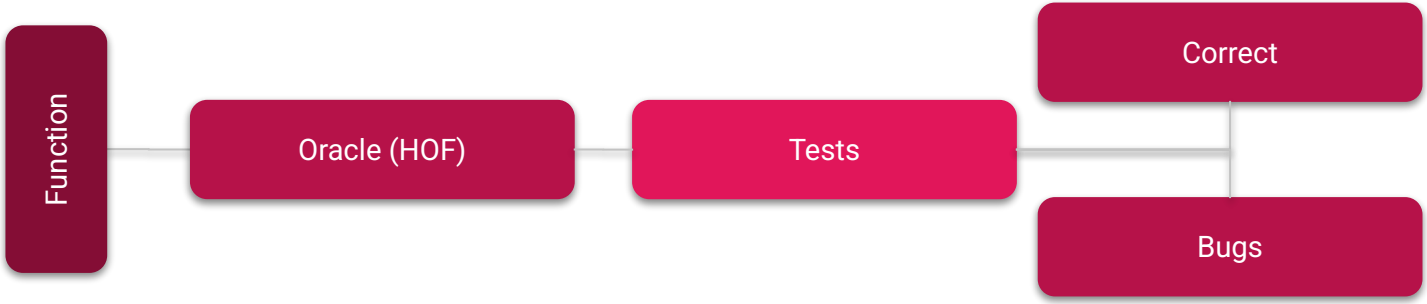
Steps to Property-Based Testing:

1. Start with a valid input to your problem
2. Run the algorithm on that input
3. Check that the result has all necessary characteristics

Oracle Functions

In this class, **oracles** are functions that determine whether or not a function correctly solves a given problem. We can determine this by performing property based testing on the outputs of the given function for various inputs.

For example, in homework 5, you write an oracle for the Preference Matching Problem.



Exercise 1: Permutations

A function **genArray**(n: number): number[][] is supposed to generate an n × n array of numbers such that each row and column is a permutation of the numbers from 0 to n-1. Assume n is nonnegative.

Write an oracle that accepts the function genArray as input.

Use higher-order functions when appropriate. Try to write your implementation in $O(n^2)$.

Solution 1: Permutations

Write an oracle that accepts the function genArray as input.

```
function oracle(genArray) {
     let n = Math.floor(Math.random()*1000);

}
```

Solution 1: Permutations

Write an oracle that accepts the function genArray as input.

```
function oracle(genArray) {
      let n = Math.floor(Math.random()*1000);
      let a = genArray(n);

}
```

Solution 1: Permutations

Write an oracle that accepts the function genArray as input.

```
function oracle(genArray) {
      let n = Math.floor(Math.random()*1000);
      let a = genArray(n);
      const valid = x => x % 1 === 0 && 0 <= x && x < n;

}
```

Solution 1: Permutations

Write an oracle that accepts the function genArray as input.

```
function oracle(genArray) {
      let n = Math.floor(Math.random()*1000);
      let a = genArray(n);
      const valid = x => x % 1 === 0 && 0 <= x && x < n;
      function isPerm(p) {

      }

}
```

Solution 1: Permutations

Write an oracle that accepts the function genArray as input.

```
function oracle(genArray) {
      let n = Math.floor(Math.random()*1000);
      let a = genArray(n);
      const valid = x => x % 1 === 0 && 0 <= x && x < n;
      function isPerm(p) {
            assert(p.length === n, 'length is n');
      }

}
```

Solution 1: Permutations

Write an oracle that accepts the function genArray as input.

```
function oracle(genArray) {
      let n = Math.floor(Math.random()*1000);
      let a = genArray(n);
      const valid = x => x % 1 === 0 && 0 <= x && x < n;
      function isPerm(p) {
            assert(p.length === n, 'length is n');
            assert(p.every(valid),'valid numbers');

      }

}
```

Solution 1: Permutations

Write an oracle that accepts the function genArray as input.

```
function oracle(genArray) {
      let n = Math.floor(Math.random()*1000);
      let a = genArray(n);
      const valid = x => x % 1 === 0 && 0 <= x && x < n;
      function isPerm(p) {
            assert(p.length === n, 'length is n');
            assert(p.every(valid),'valid numbers');
            const f = Array(n).fill(1);
            assert(p.every(e => --f[e] === 0), 'no duplicates');
      }

}
```

Note: We're checking for duplicates in a single linear scan which takes O(n) time.
This approach works for n = 0 and an empty array.

Solution 1: Permutations

Write an oracle that accepts the function genArray as input.

```
function oracle(genArray) {
      let n = Math.floor(Math.random()*1000);
      let a = genArray(n);
      const valid = x => x % 1 === 0 && 0 <= x && x < n;
      function isPerm(p) {
            assert(p.length === n, 'length is n');
            assert(p.every(valid),'valid numbers');
            const f = Array(n).fill(1);
            assert(p.every(e => --f[e] === 0), 'no duplicates');
      }
      assert(a.length === n, 'array has n rows');

}
```

Solution 1: Permutations

Write an oracle that accepts the function genArray as input.

```
function oracle(genArray) {
      let n = Math.floor(Math.random()*1000);
      let a = genArray(n);
      const valid = x => x % 1 === 0 && 0 <= x && x < n;
      function isPerm(p) {
            assert(p.length === n, 'length is n');
            assert(p.every(valid),'valid numbers');
            const f = Array(n).fill(1);
            assert(p.every(e => --f[e] === 0), 'no duplicates');
      }
      assert(a.length === n, 'array has n rows');
      a.forEach(isPerm); // each row is a permutation

}
```

a is length n, and we call isPerm (an O(n) function) for each element of a. This will take $O(n^2)$ time

Solution 1: Permutations

Write an oracle that accepts the function genArray as input.

```
function oracle(genArray) {
      let n = Math.floor(Math.random()*1000);
      let a = genArray(n);
      const valid = x => x % 1 === 0 && 0 <= x && x < n;
      function isPerm(p) {
            assert(p.length === n, 'length is n');
            assert(p.every(valid),'valid numbers');
            const f = Array(n).fill(1);
            assert(p.every(e => --f[e] === 0), 'no duplicates');
      }
      assert(a.length === n, 'array has n rows');
      a.forEach(isPerm); // each row is a permutation
      for (let k = 0; k < n; ++k) { isPerm(a.map(r => r[k])); } // each column is a permutation
}
```

Again, this will take $O(n^2)$ time

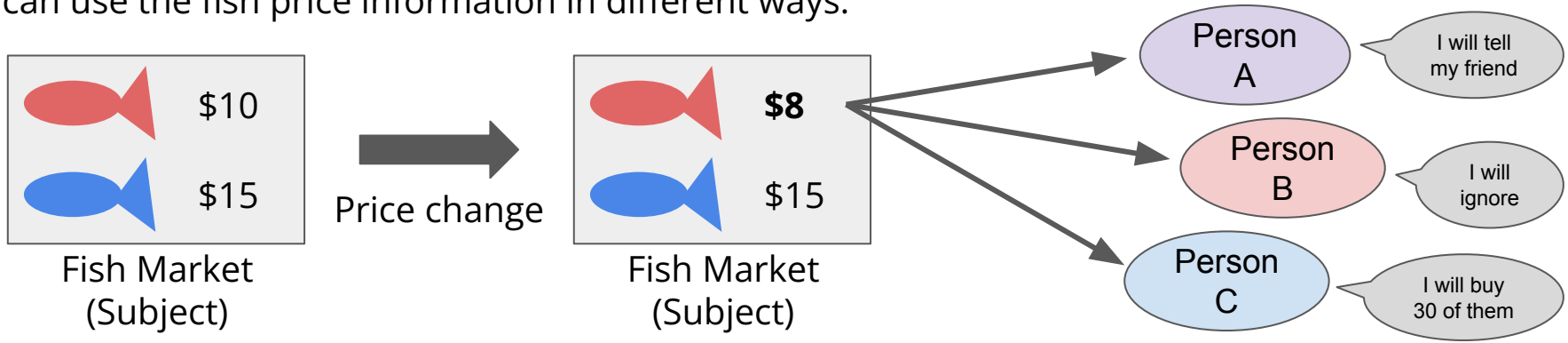
Note: A different approach, where we check directly that every number from 0 to n – 1 is included requires a linear scan for each number. That would be $O(n^2)$ per row/column and $O(n^3)$ overall. Do not do this!

Observer Pattern Review

The Observer design pattern describes two main components, Subjects and Observers.

Subjects are like publishers, which broadcast information to their “audience”. The members of this audience are Observers. Observers subscribe to Subjects in order to receive the broadcasted information.

For example, I could have a Subject for the price of different fish in a market. Each time the price of a fish changes, I will broadcast the new price to anyone who is interested. Specifically, people (Observers) can indicate their interest by subscribing to my Subject, and can use the fish price information in different ways.



Observer Pattern Review

```
interface Subject {
  // adds the given subscriber to the set of subscribers
  subscribe(subscriber: Observer): void;

  // removes the given subscriber from the set of subscribers
  unsubscribe(subscriber: Observer): void;
}
```

The Observer design pattern is defined differently in many contexts, but in this class we will assume Subjects have the above interface. Observers do not have a set interface, and will be defined based on the context.

We will typically assume both Subjects and Observers to be objects, but there are instances where it can make more sense for Observers to be just functions.

Context of Today's Exercises

You will be implementing a Subject and two Observers for handling different (computer) mouse events. Specifically, mouse clicks and mouse drags. Mouse clicks are described by the position (coordinate) clicked, and mouse drags are described by the start and end positions when the mouse is dragged.

```
interface MouseSubject {
  // adds an Observer to the set of subscribers
  subscribe(subscriber: MouseObserver): void;

  // removes an Observer from the set of subscribers
  unsubscribe(subscriber: MouseObserver): void;
}
```

Context of Today's Exercises

The MouseObserver interface has a readonly property **screen** that describes which screen this observer belongs to. Mouse events (both clicking and dragging) take place on a specific screen, and a MouseObserver should only receives updates for their corresponding screen.

Think of having 2 monitors connected to your laptop; in total you would have 3 screens where your mouse can click or drag.

```
interface MouseObserver {
  readonly screen: number;

  // update observer with new mouse click coordinate
  update_coordinate(coord: Coordinate): void;

  // update observer with new mouse drag event
  update_drag(start: Coordinate, end: Coordinate): void;
}
```

Assume screen numbers start at 0 and count upwards. I.e. if there are 3 screens total, the valid screen numbers would be 0, 1, 2.

Coordinate is defined as {x: number, y: number}. You can also assume that both x and y are non-negative.

Exercise 2: Observers

- Create a class **MouseEvents** that is a MouseSubject, implementing the following:
  - `constructor(num_screens: number)`
    - Initializes number of screens of MouseEvents and any other attributes
  - `subscribe(subscriber: MouseObserver): void`
    - Adds the given observer to a set of observers
  - `unsubscribe(subscriber: MouseObserver): void`
    - Removes the given observer if they are subscribed
  - `mouse_click(screen: number, coord: Coordinate): void`
    - Updates all observers for the given screen with the mouse click Coordinate
  - `mouse_drag(screen: number, start: Coordinate, end: Coordinate): void`
    - Updates all observers for the given screen with the start and end coordinates of the mouse drag
- Use the definitions at the top of the starter code! Try to reduce code duplication and think about how to broadcast updates efficiently.

Exercise 2: Solution

```
class MouseEvents implements MouseSubject {
  private _num_screens: number;
  private _subscribers: Set<MouseObserver>[];

  constructor(num_screens: number) {
   this._num_screens = num_screens;
   this._subscribers = [];
   // ...
  }

  // ...
}
```

We declare two private attributes, one to store the number of screens and one to store our subscribers. Since we want to update subscribers based on which screen they are subscribed to, we will store subscribers in different sets depending on their screen.

Exercise 2: Solution

```
class MouseEvents implements MouseSubject {
  private _num_screens: number;
  private _subscribers: Set<MouseObserver>[];

  constructor(num_screens: number) {
   this._num_screens = num_screens;
   this._subscribers = [];
   for (let i = 0; i < num_screens; i++) {
    const new_set = new Set<MouseObserver>();
    this._subscribers.push(new_set);
   }
  }

  // ...
}
```

We initialize this array of sets by creating a new (empty) set for each screen number.

Exercise 2: Solution

```
class MouseEvents implements MouseSubject {
  private _num_screens: number;
  private _subscribers: Set<MouseObserver>[];

  private _is_valid_screen(screen: number): boolean {
    return screen >= 0 && screen < this._num_screens;
  }

  subscribe(subscriber: MouseObserver): void {
    if (this._is_valid_screen(subscriber.screen)) {
      this._subscribers[subscriber.screen].add(subscriber);
    }
  }
  // ...
}
```

We will define a private helper method to check if a given screen number is valid for this Observable. We then define the subscribe method, adding the given subscriber to the corresponding set if their screen number is valid.

Exercise 2: Solution

```
class MouseEvents implements MouseSubject {
  private _num_screens: number;
  private _subscribers: Set<MouseObserver>[];

  unsubscribe(subscriber: MouseObserver): void {
    if (this._is_valid_screen(subscriber.screen)) {
      this._subscribers[subscriber.screen].delete(subscriber);
    }
  }

  // ...
}
```

We follow a very similar approach for unsubscribe. Note that for Sets in JS/TS, .add() adds an element to a set, and does nothing if it already exists in the set. Meanwhile, .delete() removes an element from a set, and does nothing if it is not in the set.

This would be a great place to reduce code duplication! See the solutions file for more.

Exercise 2: Solution

```
class MouseEvents implements MouseSubject {
  private _num_screens: number;
  private _subscribers: Set<MouseObserver>[];

  mouse_click(screen: number, coord: Coordinate): void {
    if (this._is_valid_screen(screen)) {
      for (const subscriber of this._subscribers[screen]) {
        subscriber.update_coordinate(coord);
      }
    }
  }
  // ...
}
```

For mouse_click, we must first check if the given screen is valid. If it is, then **this._subscribers[screen]** will give us a set of all subscribers who we should update. We then go through each of these and call the corresponding update method, passing in the coordinate the mouse was clicked at.

Exercise 2: Solution

```
class MouseEvents implements MouseSubject {
  private _num_screens: number;
  private _subscribers: Set<MouseObserver>[];

  mouse_drag(screen: number, start: Coordinate, end: Coordinate): void {
    if (this._is_valid_screen(screen)) {
      for (const subscriber of this._subscribers[screen]) {
        subscriber.update_drag(start, end);
      }
    }
  }
}
```

We follow a very similar approach for mouse_drag. Again, note that this is a great place to reduce code duplication! See the solutions file for more (posted after today).